# Passermark – Kommandozeile

Anleitung zu `passermark-cli` mit Fallbeispielen · Passermark 1.14.2

Mit der Kommandozeile laufen die schweren Funktionen von Passermark ohne Oberfläche: in Skripten, für ganze Ordner auf einmal oder später automatisch im Hintergrund. Alles, was hier steht, macht dasselbe wie die entsprechenden Dialoge im Programm.

## 1 · Aufruf

```
passermark-cli <auftrag> <eingabe.pdf> <ausgabe.pdf> [optionen]
```

| System | So heißt der Befehl |
|---|---|
| Linux (installiert mit install.sh) | `passermark-cli …` |
| Linux (AppImage) | `./Passermark-*-x86_64.AppImage --cli …` |
| Windows | `"C:\Program Files\Passermark\passermark-cli.exe" …` |
| Windows (Suchpfad bei der Installation angehakt) | `passermark-cli …` |

Windows: Im Setup gibt es (ab 1.7) das Häkchen „Kommandozeile passermark-cli in jeder Eingabeaufforderung verfügbar machen“. Dann genügt `passermark-cli` – in einem **neu geöffneten** Fenster von Eingabeaufforderung oder PowerShell. Ohne Häkchen immer den vollen Pfad in Anführungszeichen angeben.

Die Eingabedatei wird nie verändert. Die Ausgabe entsteht erst, wenn alles fertig ist – nach einem Fehler oder Abbruch bleibt keine halbe Datei liegen. Eingabe und Ausgabe dürfen nicht dieselbe Datei sein.

### Die neun Aufträge

| Auftrag | Was er macht |
|---|---|
| `cutcontour` | Schnittlinie für Schneideplotter (Sonderfarbe CutContour), optional mit Überfüller |
| `separate` | Motive erkennen, jedes Motiv als eigene Seite |
| `manip` | CMYK-Umwandlung mit ICC-Profil und/oder Beschneiden auf ein Format |
| `repair` | Reparieren, für Druck optimieren, verkleinern, PDF/A, Passwort |
| `preflight_fix` | Ebenen festschreiben, Schriften einbetten, Text in Pfade, Transparenzen reduzieren |
| `impose` | Ausschießen wie im Druckdialog: Broschüre und Lagen, Nutzen, Mehrere pro Blatt, Poster |
| `vdp` | Variable Daten: Nummern, Texte aus CSV, QR-Codes, Code 128, EAN-13 – jede Seite der nächste Datensatz |
| `split` | Seiten teilen: halbieren oder Raster, z. B. als Doppelseiten exportierte Broschüre in Einzelseiten |
| `shrink` | Dateigröße verkleinern: verlustfreie Maßnahmen einzeln schaltbar, optional Bilder herunterrechnen |

### Optionen

| Option | Bedeutung |
|---|---|
| `--set name=wert` | eine Einstellung setzen; mehrfach möglich. Zahlen, `true`/`false` und Listen werden erkannt, alles andere ist Text. Verschachtelt mit Punkt: `--set detect.tolerance=40` |
| `--preset datei.json` | Einstellungen aus einer Datei laden (siehe Abschnitt 3). `--set` überschreibt einzelne Werte daraus |

| Option | Bedeutung |
|---|---|
| `--preset "Name"` | ein im Programm gespeichertes Preset verwenden (siehe Abschnitt 3) |
| `--pages 1,3-5` | nur diese Seiten bearbeiten (wo sinnvoll) |
| `--quiet` | keine Fortschrittsanzeige |
| `--json` / `--json-progress` | Fortschritt und Ergebnis als JSON-Zeilen – für andere Programme |
| `list` | alle Aufträge anzeigen: `passermark-cli list` |
| `settings` | alle Einstellungen eines Auftrags mit Standardwerten als JSON – die Vorlage für eigene Presets |
| `presets [auftrag]` | die im Programm gespeicherten Presets auflisten |
| `beispiele [ordner]` | Beispiele für Variable Daten in den Benutzerordner kopieren (Standard: `Passermark-Beispiele`) und als Presets „Beispiel 1 …“ eintragen |
| `datenarten` | Datenarten für Tabellen (Visitenkarte, WLAN, E-Mail, Code 128, EAN-13 …) mit ihren Spalten |
| `datenvorlage [datei.csv]` | leere Tabelle mit den passenden Spalten und einer Beispielzeile |
| `datencheck [art]` | Tabelle prüfen – meldet Zeile und Fehler; Rückgabe 1 bei Problemen |
| `run`, `check`, `workflows` | Abläufe ausführen, prüfen, auflisten (Abschnitt 5) |
| `printers`, `caps` | Drucker und ihre Optionen (Abschnitt 6) |
| `inspect` | Dokument untersuchen (Abschnitt 7) |
| `version`, `schema` | für andere Programme (Abschnitt 8) |

## Rückgabewerte und Abbrechen

`0` = fertig · `1` = Fehler (z. B. Datei fehlt, kein Motiv gefunden) · `2` = falscher Aufruf · `130` = abgebrochen. **Strg+C** bricht sauber ab, ohne Ausgabedatei. In Skripten kann man so auf Fehler reagieren (Abschnitt 4).

# 2 · Die Aufträge mit Fallbeispielen

## cutcontour – Schnittlinie für den Plotter

| Einstellung | Standard | Bedeutung |
|---|---|---|
| `shape` | `contour` | Form: `contour` (folgt dem Motiv), `rect`, `rounded`, `circle`, `oval`, `hexagon`, `octagon`, `heart`, `star`, `shield`, `arch` |
| `offset_mm` | 0 | Abstand der Linie vom Motiv: + nach außen (weißer Rand), − nach innen |
| `smooth_mm` | 1,5 | Glättung der Kontur (folgt dem Motiv bewusst nicht exakt) |
| `bleed` | `true` | Überfüller erzeugen (Farbe läuft über die Schnittlinie hinaus) |
| `bleed_mm` | 2 | Breite des Überfüllers |
| `inner` | `false` | Innenkonturen (Löcher im Motiv) mitschneiden |
| `width_mm` / `height_mm` | 0 | Grundform: Größe der Schnittlinie in mm (0 = aus dem Motiv) |
| `shift_x_mm` / `shift_y_mm` | 0 | Grundform: Versatz gegenüber der Motivmitte (+ rechts / + oben) |
| `corner_mm` | 3 | Eckenradius bei `rounded` |
| `single_shape` | `true` | Grundform: eine Form um das ganze Motiv; `false` = eine je Objekt (Aufkleberbogen) |
| `per_object` | `false` | jedes Objekt auf eine eigene Seite |
| `margin_mm` | 6 | Seitenrand bei `per_object` |
| `spot` | `CutContour` | Name der Sonderfarbe – je nach RIP-Software anders |
| `stroke_pt` | 0,25 | Linienstärke |
| `dpi` | 200 | Genauigkeit; 300 dauert gut doppelt so lang |
| `detect.*` | | Motiverkennung, siehe `separate` |

### Aufkleberbogen mit Kontur und 2 mm Überfüller

Jeder Aufkleber bekommt seine Kontur, die Farbe läuft 2 mm über die Linie – kein weißer Blitzer beim Schneiden.

```
passermark-cli cutcontour bogen.pdf bogen-cut.pdf --set bleed_mm=2
```

### Logo mit Rechteck 80 × 50 mm, ohne Überfüller

Feste Größe der Schnittlinie, mittig auf dem ganzen Logo – auch wenn das Logo aus mehreren Teilen besteht.

```
passermark-cli cutcontour logo.pdf logo-cut.pdf \
    --set shape=rect --set width_mm=80 --set height_mm=50 --set bleed=false
```

### Runde Aufkleber Ø 60 mm auf einem Bogen

Jeder Aufkleber einzeln (`single_shape=false`), jeder Kreis exakt 60 mm.

```
passermark-cli cutcontour bogen.pdf rund.pdf \
    --set shape=circle --set width_mm=60 --set height_mm=60 --set single_shape=false
```

### 3 mm weißer Rand um das Motiv

Die Kontur liegt 3 mm außerhalb, ohne Überfüller bleibt der Rand weiß.

```
passermark-cli cutcontour logo.pdf logo-rand.pdf --set offset_mm=3 --set bleed=false
```

### Jeder Aufkleber auf einer eigenen Seite

Praktisch für Einzelaufträge oder wenn der Plotter Seite für Seite schneidet.

```
passermark-cli cutcontour bogen.pdf einzeln.pdf --set per_object=true --set margin_mm=5
```

### Andere Sonderfarbe für die RIP-Software

Manche Programme erwarten einen anderen Namen als „CutContour“, z. B. „Thru-cut“.

```
passermark-cli cutcontour bogen.pdf bogen-cut.pdf --set spot=Thru-cut
```

### Nur bestimmte Seiten

Seiten 1 sowie 3 bis 5 bearbeiten.

```
passermark-cli cutcontour mappe.pdf mappe-cut.pdf --pages 1,3-5
```

## separate – Motive als einzelne Seiten

| Einstellung | Standard | Bedeutung |
|---|---|---|
| `margin_mm` | 0 | Rand um jedes Motiv (negativ = nach innen) |
| `detect.mode` | `auto` | `auto`, `transparent` (freigestellt) oder `color` (einfarbiger Hintergrund) |
| `detect.tolerance` | 28 | wie stark sich das Motiv vom Hintergrund unterscheiden muss (0–255); höher = unempfindlicher gegen Flecken |
| `detect.min_size_mm` | 5 | kleinere Teile gelten als Staub |
| `detect.gap_mm` | 1 | Teile, die näher beisammen liegen, gehören zu einem Motiv |

### Scan mit mehreren Motiven zerlegen

Jedes Motiv wird eine Seite, mit 3 mm Rand.

```
passermark-cli separate scan.pdf motive.pdf --set margin_mm=3
```

### Fleckiger Hintergrund (z. B. Altpapier)

Höhere Toleranz und größere Mindestgröße – Flecken werden nicht mehr als Motiv erkannt.

```
passermark-cli separate scan.pdf motive.pdf \
    --set detect.tolerance=40 --set detect.min_size_mm=10
```

## manip – CMYK-Umwandlung und Beschneiden

| Einstellung | Standard | Bedeutung |
|---|---|---|
| `crop` | `false` | auf ein Format bringen (Überstand beidseitig gleich) |
| `crop_scale` | `true` | zuerst skalieren, bis das Format ganz gefüllt ist (A4 → A6 = 50 %), dann den Überstand der anderen Kante abschneiden; `false` = nur abschneiden (Dateien mit Anschnitt) |
| `crop_inset_mm` | 0 | vorher rundum so viel abschneiden (weiße Ränder der Vorlage) |
| `crop_size` | A4 | A0–A7, B4, B5, C4–C6, DL, SRA3, A3+, Letter, Legal, Tabloid, „Visitenkarte 85×55“, „Visitenkarte 90×50“, „Quadrat 210“, „Quadrat 148“ oder `custom` |
| `crop_w_mm` / `crop_h_mm` | 210 / 297 | Maße bei `custom` |
| `crop_follow` | `true` | Hoch-/Querformat der Seite folgen |
| `cmyk` | `false` | nach CMYK umwandeln (braucht Ghostscript) |
| `target` | | Pfad zum CMYK-Zielprofil (.icc), z. B. ISOcoated_v2_eci.icc |
| `cmyk_mode` | `rgb_only` | `rgb_only` (vorhandenes CMYK bleibt), `all` (auch CMYK umrechnen, Quellprofil in `source`), `gray` |
| `intent` | `relative` | `perceptual`, `relative`, `saturation`, `absolute` |

| Einstellung | Standard | Bedeutung |
|---|---|---|
| gray_to_k | true | Grau/Schwarz nur mit K drucken |

### Alle Seiten auf A5 bringen

A4 wird auf 70,7 % verkleinert; ein schmaleres Motiv füllt die Breite, oben und unten wird gleich viel abgeschnitten.

```
passermark-cli manip flyer.pdf flyer-a5.pdf --set crop=true --set crop_size=A5
```

### Datei mit 3 mm Anschnitt auf Endformat (nur abschneiden)

```
passermark-cli manip flyer-anschnitt.pdf flyer-a4.pdf \
    --set crop=true --set crop_size=A4 --set crop_scale=false
```

### Eigenes Format 100 × 150 mm

```
passermark-cli manip foto.pdf foto-10x15.pdf \
    --set crop=true --set crop_size=custom --set crop_w_mm=100 --set crop_h_mm=150
```

### Druckdaten nach CMYK (ISO Coated v2)

Das Profil gibt es kostenlos bei der ECI (eci.org). Pfad an deinen Speicherort anpassen.

```
passermark-cli manip druck.pdf druck-cmyk.pdf \
    --set cmyk=true --set target=/pfad/ISOcoated_v2_eci.icc
```

## repair – Reparieren, optimieren, PDF/A, Passwort

| Einstellung | Standard | Bedeutung |
|---|---|---|
| mode | print | repair (nur reparieren), print (für Weitergabe und Druck, Schriften einbetten), screen (für Mail verkleinern, 150 dpi), pdfa (PDF/A-2b, Archiv/Behörden) |
| password | | Passwort der Eingabedatei, falls geschützt |
| new_password | | neues Passwort für die Ausgabe (leer = keins); nicht bei pdfa – PDF/A verbietet Verschlüsselung, das Passwort wird dann weggelassen |
| allow_print | true | mit Passwort: Drucken trotzdem erlauben |

### Kaputtes PDF für den Druck aufbereiten

```
passermark-cli repair kaputt.pdf ok.pdf --set mode=print
```

### Für Mail verkleinern

```
passermark-cli repair katalog.pdf katalog-klein.pdf --set mode=screen
```

### Archivieren als PDF/A

Für Archiv, Behörden und E-Rechnungs-Anhänge.

```
passermark-cli repair vertrag.pdf vertrag-a.pdf --set mode=pdfa
```

### Mit Passwort schützen

Öffnen nur mit Passwort, Drucken bleibt erlaubt.

```
passermark-cli repair angebot.pdf angebot-geschuetzt.pdf --set new_password=geheim
```

## preflight_fix – Problem-PDFs druckfest machen

| Einstellung | Standard | Bedeutung |
|---|---|---|
| fix | flatten_layers | flatten_layers (Ebenen festschreiben – gedruckt wird genau das Sichtbare), embed_fonts, outline_text (Text in Pfade), flatten_transparency, flatten_softmask (transparente Verläufe in deckende Bilder – gegen schwarze Flächen am RIP, ohne Ghostscript) |
| font_map | {} | bei embed_fonts: Ersatzschriften {Schriftname: Pfad} |
| dpi | 300 | bei flatten_transparency und flatten_softmask: Auflösung der gerasterten Bereiche |

### Plan mit Ebenen: nur das Sichtbare drucken

```
passermark-cli preflight_fix plan.pdf plan-fest.pdf --set fix=flatten_layers
```

### Alter RIP: Transparenzen reduzieren

```
passermark-cli preflight_fix plakat.pdf plakat-flach.pdf \
    --set fix=flatten_transparency --set dpi=300
```

embed_fonts, outline_text, flatten_transparency, repair und die CMYK-Umwandlung brauchen Ghostscript. Windows-Setup und AppImage bringen es mit; bei der Linux-Installation installiert es install.sh.

### shrink – Dateigröße verkleinern

Jede Maßnahme ist ein eigenes Häkchen – macht eine Probleme, einfach mit --set name=false abschalten. Ohne Ghostscript. Ist das Ergebnis nicht kleiner, wird die Datei unverändert übernommen.

| Einstellung | Standard | Bedeutung |
|---|---|---|
| rewrite | true | neu schreiben: alte Speicherstände weg, komprimierte Objekt-Streams, ZIP stärker |
| unreferenced | true | nicht mehr verwendete Ressourcen entfernen |
| dedupe | true | doppelt gespeicherte Bilder, Schriften, ICC-Profile zusammenlegen |
| private | true | Illustrator-/Photoshop-Bearbeitungsdaten (PieceInfo) entfernen |
| thumbs | true | eingebettete Seitenvorschaubilder entfernen |
| pack | true | Bilder verlustfrei packen (ZIP mit Prädiktor), ZIP um JPEG entfernen – nur wenn kleiner |
| bits16 | false | 16-Bit-Bilder auf 8 Bit |
| attachments | false | eingebettete Dateien (Anhänge) entfernen |
| hidden | false | Inhalte ausgeblendeter Ebenen entfernen |
| downsample | false | Bilder über target_dpi × threshold (gemessen an der größten Platzierung) auf target_dpi herunterrechnen |
| target_dpi | 300 | Ziel-Auflösung beim Herunterrechnen |
| threshold | 1.5 | erst ab Ziel × Schwelle (1.5 bei 300 dpi = ab 450 dpi) |
| photos_jpeg | false | Fotos, die als ZIP gespeichert sind, als JPEG speichern (nur echte Fotos, nur wenn deutlich kleiner) |
| jpeg_quality | 92 | JPEG-Qualität beim Herunterrechnen und für photos_jpeg |

### Nur verlustfrei verkleinern

Das Druckbild bleibt pixelgleich.

```
passermark-cli shrink flyer.pdf flyer-klein.pdf
```

### Plakat: Bilder auf 150 dpi

Bearbeitungsdaten behalten.

```
passermark-cli shrink plakat.pdf plakat-klein.pdf \
    --set downsample=true --set target_dpi=150 --set private=false
```

## impose – Broschüren, Lagen und Nutzen ausschießen

Dasselbe wie der Druckdialog, aber als PDF-Datei: jede Seite der Ausgabe ist ein Druckbogen. Das Bogenformat steht in `sheet`. Alle Einstellungen des Druckdialogs zeigt `passermark-cli settings impose`; die wichtigsten:

| Einstellung | Standard | Bedeutung |
|---|---|---|
| `sheet` | `A4` | Bogen: A0–A6, A3+, SRA3, B4–B6, LETTER, LEGAL, TABLOID oder `custom` mit `sheet_w_mm`/`sheet_h_mm` |
| `handling` | `size` | `booklet` (Broschüre), `multiple` (mehrere pro Blatt), `poster` |
| `booklet_kind` | `saddle` | `saddle` = Sammelheftung (alle Bögen ineinander), `grouped` = Lagen aus mehreren Bögen, `stack` = Einzelbögen (Klebebindung) |
| `booklet_per_sig` | 4 | bei `grouped`: Bögen je Lage (4 Bögen = 16 Seiten) |
| `booklet_creep_mm` | 0 | Bundzug je Bogen, etwa die Papierstärke (0.1 bei 80 g/m²) |
| `booklet_blanks` | `end` | Leerseiten `end` oder `before_back` (Rückseite bleibt letzte Seite) |
| `booklet_blank_text` | – | Hinweistext auf aufgefüllten Leerseiten |
| `booklet_fold_marks` | `false` | Falzmarken |
| `booklet_reg_marks` | `false` | Passermarken |
| `booklet_collation_marks` | `false` | Flattermarken am Rücken jeder Lage |
| `booklet_gutter_mm` | 0 | Bundsteg (Abstand zum Falz) |
| `step_repeat` | `false` | Nutzen: eine Seite mehrfach je Bogen; `sr_by=long`/`short` + `sr_mm` für die Nutzengröße |
| `bleed_mm`, `crop_marks` | 0, `false` | Anschnitt (gespiegelt) und Schnittmarken |

### Heft A5 auf A4 (Sammelheftung)

Leerseiten vor der Rückseite, Falzmarken.

```
passermark-cli impose heft.pdf heft-bogen.pdf --set handling=booklet --set sheet=A4 \
    --set booklet_blanks=before_back --set booklet_fold_marks=true
```

### Dickes Buch in Lagen zu 16 Seiten

4 Bögen je Lage, Bundzug 0,1 mm, Flattermarken.

```
passermark-cli impose buch.pdf buch-bogen.pdf --set handling=booklet --set sheet=A3 \
    --set booklet_kind=grouped --set booklet_per_sig=4 --set booklet_creep_mm=0.1 \
    --set booklet_collation_marks=true
```

### Flyer A4 als A6 8× auf SRA3

Lange Kante 148 mm, 2 mm Anschnitt, Schnittmarken.

```
passermark-cli impose flyer.pdf flyer-sra3.pdf --pages 1 --set sheet=SRA3 \
    --set step_repeat=true --set sr_by=long --set sr_mm=148 \
    --set bleed_mm=2 --set crop_marks=true
```

Druck-Presets aus dem Programm gelten hier direkt – samt Papierformat: im Druckdialog einstellen, oben **Speichern…**, dann `--preset "Name"`. Die Druckreihenfolge der Bögen ist für beidseitigen Druck über die **kurze Kante** ausgelegt. Hat das PDF ein Endformat mit Anschnitt (TrimBox, z. B. aus InDesign mit „Anschnitt verwenden“), wird das Endformat platziert und der Anschnitt aus dem Dokument genommen – `sr_mm` meint dann das Endformat.

## vdp – Variable Daten: Nummern, Namen, QR- und Barcodes

Legt Felder auf die Vorlage – jede Seite bekommt den nächsten Datensatz. Datensätze kommen aus einer CSV-Datei (erste Zeile = Spaltennamen, Trennzeichen ; , oder Tab werden erkannt) oder aus einer reinen Nummerierung (`count`). Inhalte sind Vorlagen: `{{Spalte}}`, `{{nr}}` (formatierte Nummer), `{{i}}` (laufend ab 1). Alles bleibt Vektor.

Die Felder legt man am einfachsten im Programm an (Arbeitsbereich Variable Daten, Strg+Umschalt+D, Felder mit der Maus platzieren, Vorschau je Datensatz) und speichert ein Preset. Die Kommandozeile nimmt dann das Preset und ändert nur, was sich je Auftrag ändert:

| Einstellung | Standard | Bedeutung |
|---|---|---|
| `csv_path` | – | CSV-Datei; leer = nur Nummerierung |
| `count` | 1 | ohne CSV: so viele Kopien |
| `records` | – | nur diese Datensätze, z. B. 1-50 |
| `numbering.start` / `.step` / `.digits` | 1 / 1 / 0 | Start, Schrittweite, mit Nullen auf so viele Stellen |
| `numbering.prefix` / `.suffix` | – | Text vor/nach der Nummer |
| `numbering.check` | `none` | `luhn`, `ean`, `mod11` – Prüfziffer anhängen |
| `numbering.continue_key` | – | Zählername: der nächste Auftrag zählt dort weiter |
| `reverse` | `false` | rückwärts (Abreißblock: oberstes Blatt höchste Nummer) |
| `pages_per_record` | 1 | bei each: so viele Seiten je Datensatz – 2 = Vorder- und Rückseite |
| `fields[].pages` | – | Feld nur auf diesen Vorlagenseiten: `1,3-4`, `ungerade`, `gerade`, `ungerade 1-50` |
| `order` | each | each = jede Seite der nächste Datensatz (einseitige Vorlage: je Datensatz eine Seite), `record` = je Datensatz alle Seiten, page = wie record, Seite für Seite |
| `log_path` | – | Code-Protokoll als CSV (Ausgabeseite, Datensatz, Werte) |
| `placeholders` | `false` | `{{…}}` im PDF als Feldposition übernehmen, Platzhaltertext entfernen |
| `fields[].qr_type` | `text` | QR-Feld: `vcard`, `wifi`, `email`, `url`, `phone`, `sms`, `event`, `geo` – Inhalt aus den Tabellenspalten (im Preset; im Programm „Art des QR-Codes“) |
| `fields[].qr_map` | `{}` | Zuordnung Angabe → CSV-Spalte, z. B. `{"Position": "Title"}`; `-` = leer lassen; nicht genannte werden automatisch erkannt (auch Outlook-/Google-Exporte) |

Fertige Beispiele mit Vorlage, CSV und Presets (QR-Code, Code 128, EAN-13) holt `passermark-cli beispiele`; ausführlich erklärt in der Anleitung „Variable Daten“ (Hilfe → Variable Daten – Anleitung).

### 500 nummerierte Tickets

Preset „Tickets“ im Programm angelegt (Feld mit {{nr}}).

```
passermark-cli vdp ticket.pdf tickets.pdf --preset "Tickets" --set count=500
```

### Namensschilder aus einer Gästeliste

Felder {{Name}} und QR-Code mit {{Name}}; Protokoll dazu.

```
passermark-cli vdp schild.pdf schilder.pdf --preset "Namensschilder" \
    --set csv_path=gaeste.csv --set log_path=codes.csv
```

### Visitenkarten mit QR-Code aus einer Kontaktliste

Leere Tabelle anlegen, in Excel ausfüllen, prüfen, erzeugen. Das Preset hat ein QR-Feld mit Art „Visitenkarte“ (qr_type vcard).

```
passermark-cli datenvorlage vcard kontakte.csv
passermark-cli datencheck kontakte.csv
passermark-cli vdp karte.pdf karten.pdf --preset "Visitenkarte" --set csv_path=kontakte.csv
```

### Jede Karte mit eigenem Code 8× auf SRA3, schneiden und stapeln

Erst die Daten, dann ausschießen – je Nutzen die nächste Seite, Stapel bleiben fortlaufend.

```
passermark-cli impose tickets.pdf tickets-sra3.pdf --set sheet=SRA3 \
    --set step_repeat=true --set sr_sequence=true --set sr_stack=stack \
    --set crop_marks=true
```

### split – Seiten teilen (halbieren, Raster, Doppelseiten)

Teilt Seiten in Einzelseiten – verlustfrei, der Inhalt wird nicht verändert. Hat die Seite ein Endformat, wird darin geteilt und der Anschnitt bleibt außen. Gedrehte Seiten werden so geteilt, wie man sie sieht. Im Programm: Seiten markieren, Rechtsklick → Teilen → halbieren; oder Seiten → Seiten teilen … bzw. Doppelseiten teilen.

| Einstellung | Standard | Bedeutung |
|---|---|---|
| `mode` | `halves_v` | `halves_v` = links \| rechts, `halves_h` = oben / unten, `grid` = Raster |
| `cols` / `rows` | 2 / 2 | nur bei `grid`: Spalten × Zeilen |
| `only_spreads` | `false` | nur Seiten teilen, die doppelt so breit (bzw. Raster-mal so groß) wie die übrigen sind |
| `rtl` | `false` | Reihenfolge rechts nach links (Bindung rechts) |
| `use_trim` | `true` | im Endformat teilen, Anschnitt außen behalten |

### Broschüre als Doppelseiten exportiert

Seite 1 einzeln A4, dann A3-Doppelseiten: nur die Doppelseiten halbieren – ergibt fortlaufende A4-Seiten.

```
passermark-cli split broschuere.pdf einzelseiten.pdf --set only_spreads=true
```

### Bogen mit 2 × 4 Karten in Einzelkarten

Raster auf Seite 1.

```
passermark-cli split bogen.pdf karten.pdf --set mode=grid --set cols=2 --set rows=4 --pages 1
```

# 3 · Presets: Einstellungen als Datei

Wer dieselben Einstellungen immer wieder braucht, legt sie einmal in einer Datei ab. Die Vorlage mit allen Einstellungen und ihren Standardwerten liefert `settings`.

```
passermark-cli settings cutcontour > sticker.json
```

Dann die Datei in einem Texteditor anpassen – nur die Werte hinter dem Doppelpunkt ändern:

```
{
  "job": "cutcontour",
  "settings": {
    "shape": "rounded",
    "corner_mm": 4,
    "offset_mm": 2,
    "bleed_mm": 2,
    ...
  }
}
```

Verwenden – einzelne Werte lassen sich trotzdem noch mit `--set` ändern:

```
passermark-cli cutcontour bogen.pdf bogen-cut.pdf --preset sticker.json
passermark-cli cutcontour bogen.pdf bogen-cut.pdf --preset sticker.json --set bleed_mm=3
```

Ein Preset gehört immer zu einem Auftrag – ein CutContour-Preset kann nicht für `separate` verwendet werden. Werte mit Sonderzeichen (z. B. `font_map` mit Windows-Pfaden) trägt man am einfachsten im Preset statt mit `--set` ein.

### Presets aus dem Programm

Einfacher geht es im Programm selbst: In den Fenstern für CutContour, CMYK/Beschneiden und im Druckdialog gibt es oben die Zeile **Preset**. Einstellungen wie gewünscht setzen, **Speichern…** und einen Namen vergeben. Die Kommandozeile findet das Preset dann über seinen Namen – ohne Pfad:

```
passermark-cli presets cutcontour
passermark-cli cutcontour bogen.pdf bogen-cut.pdf --preset "Sticker rund"
```

Abgelegt sind die Presets als JSON-Dateien je Auftrag – unter Linux in `~/.config/passermark/presets/`, unter Windows in `%APPDATA%\Passermark\presets\`. Im Programm: Arbeitsbereich **Automatisierung** → **Presets** öffnet den Ordner. Druck-Presets (Broschüre, Poster, Nutzen) gehören zum Auftrag `impose`.

# 4 · Ganze Ordner auf einmal

Linux (Terminal): alle PDFs aus `eingang` schneiden, Ergebnisse nach `fertig`, Fehler werden gemeldet.

```
mkdir -p fertig
for f in eingang/*.pdf; do
    passermark-cli cutcontour "$f" "fertig/$(basename "$f" .pdf)-cut.pdf" \
        --preset sticker.json --quiet || echo "FEHLER: $f"
done
```

Windows (PowerShell):

```
$cli = "C:\Program Files\Passermark\passermark-cli.exe"
New-Item -ItemType Directory -Force -Path fertig | Out-Null
Get-ChildItem eingang\*.pdf | ForEach-Object {
    & $cli cutcontour $_.FullName "fertig\$($_.BaseName)-cut.pdf" `
        --preset sticker.json --quiet
    if ($LASTEXITCODE -ne 0) { Write-Host "FEHLER: $($_.Name)" }
}
```

# 5 · Abläufe: mehrere Schritte als JSON

Ein Ablauf ist eine JSON-Datei mit mehreren Aufträgen hintereinander – jeder Schritt arbeitet mit dem Ergebnis des vorigen. Die Ausgabe wird erst geschrieben, wenn alle Schritte geklappt haben. Abläufe schreibt man von Hand (oder lässt sie von einem Zusatzprogramm erzeugen); im Programm selbst werden keine Abläufe zusammengestellt.

```
{
  "passermark": "workflow", "version": 1,
  "name": "Visitenkarten SRA3",
  "description": "Ebenen festschreiben, 85 x 55 mm so oft wie möglich auf SRA3",
  "output": "${dir}/fertig/${stem}-sra3.pdf",
  "steps": [
    {"job": "preflight_fix", "settings": {"fix": "flatten_layers"}},
    {"job": "impose", "pages": "1-2",
     "settings": {"sheet": "SRA3", "step_repeat": true, "crop_marks": true, "bleed_mm": 3}}
  ]
}
```

| Eintrag | Bedeutung |
|---|---|
| `input` | Eingabedatei oder Liste – mehrere Dateien, Bilder und Office-Dokumente werden zuerst zu einem PDF zusammengeführt. Meist gibt man sie beim Aufruf mit `--in` an |
| `output` | Ergebnis. Fehlende Ordner werden angelegt. Darf fehlen, wenn ein Schritt druckt |
| `steps` | die Schritte: `job` wie oben, dazu `settings` (nur, was vom Standard abweicht), `preset` (Name oder Datei) und `pages` |
| `${name}` | Variablen: `--var name=wert`, dazu fest `${input}`, `${dir}` (Ordner der Eingabe), `${stem}` (Name ohne Endung), `${date}`, `${workflow_dir}` |

Unbekannte Einträge und falsche Werte sind Fehler – ein Tippfehler fällt nicht still auf den Standard zurück. check meldet alle Probleme auf einmal, ohne etwas auszuführen.

## Prüfen und ausführen

Pfade beim Aufruf gelten ab dem aktuellen Ordner, Pfade in der Datei ab deren Ordner.

```
passermark-cli check visitenkarten.json --in karten.pdf
passermark-cli run visitenkarten.json --in karten.pdf
```

## Eigene Ausgabe und Variablen

Ablauf `kunde.json`: verkleinern, dann Broschüre; die Ausgabe landet in einem Ordner je Kunde.

```
passermark-cli run kunde.json --in katalog.pdf --var kunde=Muster
passermark-cli run kunde.json heft.pdf --in katalog.pdf --var kunde=Muster
```

```
{
  "passermark": "workflow", "version": 1,
  "output": "${dir}/${kunde}/${date}-${stem}.pdf",
  "steps": [
    {"job": "shrink", "settings": {"downsample": true, "target_dpi": 300}},
    {"job": "impose", "settings": {"sheet": "A4", "handling": "booklet", "booklet_fold_marks": true}}
  ]
}
```

## Ablauf-Ordner: Abläufe beim Namen aufrufen

Liegt ein Ablauf in einem der Ablauf-Ordner, genügt sein Name (Dateiname ohne `.json`). Der eigene Ordner gilt vor dem für alle; Presets und CSV-Dateien, die der Ablauf braucht, legt man einfach daneben.

| | eigene | für alle Benutzer (Administrator) |
|---|---|---|
| Linux | `~/.config/passermark/ablaeufe/` | `/etc/passermark/ablaeufe/` |

| | eigene | für alle Benutzer (Administrator) |
|---|---|---|
| Windows | `%APPDATA%\Passermark\ablaeufe\` | `%ProgramData%\Passermark\ablaeufe\` |

```
passermark-cli workflows
passermark-cli workflows --dirs
passermark-cli run Visitenkarten --in karten.pdf
```

Den eigenen Ordner legt man einmal selbst an (Linux: `mkdir -p ~/.config/passermark/ablaeufe`); `workflows --dirs` zeigt, wo er hingehört.

# 6 · Drucken

Der Schritt `print` schickt das PDF an einen Drucker – auf demselben Weg wie der Druckdialog: Seitenhandhabung (Broschüre, Nutzen, Poster …), Farbprofil des Druckers aus der Verwaltung, automatische Fachwahl. Das PDF bleibt dabei unverändert; danach kann der Ablauf weitermachen oder speichern.

```
{
  "passermark": "workflow", "version": 1,
  "steps": [
    {"job": "print", "settings": {
      "printer": "Canon_iR-ADV_C3822i", "paper": "A4", "tray": "Fach 2",
      "duplex": "long", "copies": 2, "finishing": {"staple": "links oben"},
      "dry_run": true}}
  ]
}
```

| Einstellung | Bedeutung |
|---|---|
| `printer` | Name wie bei `passermark-cli printers`; leer = Standarddrucker |
| `paper`, `tray`, `media`, `output_bin` | Papierformat, Fach, Medium, Ausgabefach – Treiberwert oder der angezeigte Name, ein eindeutiger Teil genügt (`"Fach 2"`) |
| `duplex`, `color` | `none` / `long` / `short` bzw. `color` / `gray` – oder der Treiberwert |
| `finishing` | Finisher: `staple`, `punch`, `fold`, `saddle`, `trim`, `stacker` |
| `finisher_preset` | Finisher-Vorlage aus der Verwaltung – unter Windows der Weg zu allen Herstellerfunktionen (Original-Treiberdialog) |
| `options` | weitere Treiberoptionen roh: `{"Schlüsselwort": "Wert"}` |
| `copies`, `collate`, `reverse` | Kopien, sortiert, umgekehrte Reihenfolge |
| `layout` | Seitenhandhabung wie im Druckdialog, z. B. `{"handling": "booklet"}` |
| `dry_run` | `true` = nichts senden, nur anzeigen, was gesendet würde |

### Drucker und ihre Möglichkeiten

Alle Optionen, Fächer, Formate und Finisher eines Druckers – was dort steht, gehört in den Ablauf.

```
passermark-cli printers
passermark-cli caps "Canon_iR-ADV_C3822i"
```

Ein Wert, der zu keiner Auswahl passt, ist ein Fehler, der alle möglichen Werte nennt. Was der Treiber mit der eingetragenen Hardware nicht kann (z. B. Lochen ohne Locher), wird abgelehnt. Druckernamen mit Leerzeichen in Anführungszeichen; unter Linux hat die CUPS-Warteschlange keine Leerzeichen (`Canon_iR-ADV_C3822i`).

# 7 · Dokumente untersuchen

`inspect` zeigt Seiten, Formate, Endformat und Anschnitt je Seite, Dokumentangaben und mit `--check` die Prüfergebnisse (Schriften, Ebenen, Bilder, Farben) – dieselben Angaben wie **Datei →**

**Eigenschaften** im Programm.

```
passermark-cli inspect karten.pdf --check
```

## 8 · Für andere Programme

Andere Programme (Skripte, Watcher, Shop- und Intranet-Anbindung) verwenden Passermark nur über die Kommandozeile – mit `--json` kommt alles als JSON-Zeilen mit festen Fehlercodes (`usage`, `not_found`, `invalid`, `failed`, `cancelled`). Die genaue Beschreibung steht in `docs/cli-api.md`.

```
passermark-cli version --json
passermark-cli schema impose
passermark-cli run visitenkarten.json --in karten.pdf --json
```

## 9 · Tipps und Fehler

| Meldung / Problem | Lösung |
|---|---|
| Kein Motiv gefunden | Toleranz senken (`--set detect.tolerance=15`) oder Modus festlegen (`--set detect.mode=color`) |
| Flecken werden als Motiv erkannt | Toleranz erhöhen, `detect.min_size_mm` vergrößern |
| Teile eines Logos bekommen getrennte Konturen | `detect.gap_mm` erhöhen, z. B. 3 |
| Dauert lange | `dpi` auf 200 lassen; 300 dauert gut doppelt so lang. Große Seiten mit vielen Objekten rechnet Passermark automatisch auf mehreren Rechenkernen |
| Rechner soll nebenbei frei bleiben | Rechenkerne begrenzen: Linux `PASSERMARK_WORKERS=2 passermark-cli …`, Windows (PowerShell) `$env:PASSERMARK_WORKERS=2` davor; 1 = nicht parallel |
| Preset ist für „…“ | Das Preset gehört zu einem anderen Auftrag |
| Wert wird als Text statt Zahl gelesen | Dezimalpunkt verwenden: `bleed_mm=2.5` (nicht 2,5) |
| Leerzeichen im Dateinamen | Namen in Anführungszeichen setzen: `"mein bogen.pdf"` |
| „… ist kein Auftrag“ bei einer .json | Abläufe mit `run` starten: `passermark-cli run ablauf.json --in datei.pdf` |
| „ist ein Ablauf, kein Preset“ | Die Datei hat `steps` – mit `run` ausführen, nicht als `--preset` laden |
| Ablauf nicht gefunden | `passermark-cli workflows` zeigt alle Namen, `--dirs` die Ordner |

## 10 · Ausblick

Die Kommandozeile wächst mit Passermark. Mit den nächsten Versionen kommen dazu – jeweils mit Fallbeispielen in dieser Anleitung:

| Version | Neu in der Kommandozeile |
|---|---|
| 1.15 | Watcher: Datei in einen Ordner werfen – der zugeordnete Ablauf verarbeitet, druckt oder speichert sie; als Dienst ohne angemeldeten Benutzer |